GOLD: Types and Strings (Mon Aug 31)

Review: Java Basics

Summary of Java so far

- Program structure
    - Top-level structure is **method definitions**
        - Name of file (with `.java` extension) is up to you
        - Example: `Robot.java`
    - One or more methods
    - Must include `void main() { ... }` as one of the methods
    - Program execution begins at `main`
- Command line input and output
    - `String IO.readln(prompt)` Prints the value of `prompt` to the screen, waits for input, and then returns that value
    - `void IO.println(message)` outputs the expression `message` to the command line, converting to a `String` if needed, and starts a new line afterwards.
        - `void IO.print(message)` outputs the expression `message`, but doesn't start a new line afterwards

New and Old Style

New and Old Style for Simple Programs

The simple programs we've written so far

```
void main() {
    IO.println("Boiler Up!");
    response();
}

void response() {
    IO.println("Hammer Down!");
}
```

are a convenient shorthand for the following more verbose style:

```
public class Rally {
    public static void main(String[] args) {
        System.out.println("Boiler Up!");
        response();
    }

    public static void response() {
        System.out.println("Hammer Down!");
    }
}
```

The old style will still be used on some lab handouts this semester.

Eventually you will start writing this kind of code, after we teach you what those words mean and why you need them sometimes.

Above, we've included a second method, `response()` . A method is a named block of code that `main` runs by writing its name and parentheses (with or without arguments).

You'll learn to write your own methods later. For now, read one like a mini-program with a name.

Syntax Details

Syntax Details

- `//` for comments (one line) or `/* ... */` for comments over several lines
- White space ignored (delimits tokens)
- Semicolon (`;`) to end or separate statements
- Curly braces (`{}`) to group statements
- Some words are "reserved" and cannot be used for variable, method, or class names
- Use a `.` to separate the name of **classes** from the name of methods defined in them
- method calls like `IO.readln("What's your name? ")` use parentheses after the name that are either empty or contain **arguments** separated by commas

A Field Guide to Types

A Field Guide to Types: Some Tools and Truths

The Field Guide: four tools

1. Pretty output: `String.format`
2. Comparing Strings: `==` lies, `.equals` doesn't
3. A truth about decimals: `0.1 + 0.2`
4. Forcing (numeric) types to convert: casting

Rounding and formatting

Part 1: `String.format`: fixing `$8.5`

Last week, our register printed text like `$8.5.` But a real receipt would say `$8.50`.

```
void main() {
    double total = 8.5;
    IO.println("Total: $" + total);          // Total: $8.5
}
```

Java has a function called `String.format` that takes a **template** with holes, and then the values to fill them. The holes have to have the following special formats:

Hole	Fills with
`%.2f`	a double, rounded to 2 decimals
`%d`	an int
`%s`	a String

```
String receiptLine = String.format("%s ordered %d burgers: $%.2f", "Priya", 2, 17.0);
// Priya ordered 2 burgers: $17.00
```

Now we can send `receiptLine` to `IO.println`:

```
void main() {
    double total = 8.5;
    String receiptLine = String.format("Total: $%.2f", total);
    IO.println(receiptLine);          // Total: $8.50
    IO.println("Goodbye!");
}
```

Shorter version : `IO.println(String.format("Total: $%.2f", total));`

In Java, `String` is a type and also a class that contains methods we can use directly.

Pause and predict (poll)

Q. What does this actually do? `String.format("Total: $%.2f", total);`

- **A.** Prints the formatted/rounded total to the screen
- **B.** Builds and returns a new String, which you can then print
- **C.** Changes the value in `total` to have 2 decimal places
- **D.** Rounds `total` permanently to 8.5

Poll Reveal: String.format

Question *Submitted Sep 30th 2026 at 5:28:16 pm*

What does this actually do? `String.format("Total: $%.2f", total);`

- ( ) Prints the formatted/rounded total to the screen
- (•) Builds and returns a new String, which you can then print
- ( ) Changes the value in total to have 2 decimal places
- ( ) Rounds total permanently to 8.5

Reference: printf

The older way:

There's an even shorter version. Use `System.out.printf` to do both formatting and printing in one call:

```
void main() {
    double total = 26.5;
    System.out.printf("Total: $%.2f\n", total);  // Total: $26.50
    IO.println("Goodbye!");
}
```

The strange characters `\n` above are an **escape sequence** that called the **newline** which starts the next terminal output on the next line.



There's a big gotcha with `printf` : **it does not end the current line automatically**, so you need to include `\n` when you want the next output to go on the next line (which is most of the time, except when you're asking the user for input).

More format Strings

Why do we have various format specifiers to remember?

Hole	Fills with
`%.2f`	a double, rounded to 2 decimals
`%d`	an int
`%s`	a String

Pause and Predict:

What does this print?

```
IO.println(String.format("Total: %d", 26.5));
```

```
void main() {
    IO.println(String.format("Total: %d for %d burgers for %s", 26.5, 6, "Sripath"));
}
```

▶ Expand

Pause and predict (poll)

Q. Which format specifier fills in a `double`, rounded to the nearest whole number?

- **A.** `%d`
- **B.** `%0f`
- **C.** `%.0f`
- **D.** `%s`
- **E.** `%2f`

Format String Practice

Why do we have various format specifiers to remember?

Hole	Fills with
`%.2f`	a double, rounded to 2 decimals
`%d`	an int
`%s`	a String

Think and Share

What format specifier do you think you might try if you wanted to print a `double` decimal rounded to the nearest whole number?

Don't look it up; try to generalize from the table above. I'm not including a code block here on purpose.

Question

Which format specifier fills in a `double`, rounded to the nearest whole number?

- `%d`
- `%.2f`
- `%2f`
- `%s`
- %.0f

String Equality

Part 2: When Comparing Strings, `==` lies

Let's examine this code:

```
void main() {
    String word = IO.readln("Password? ");

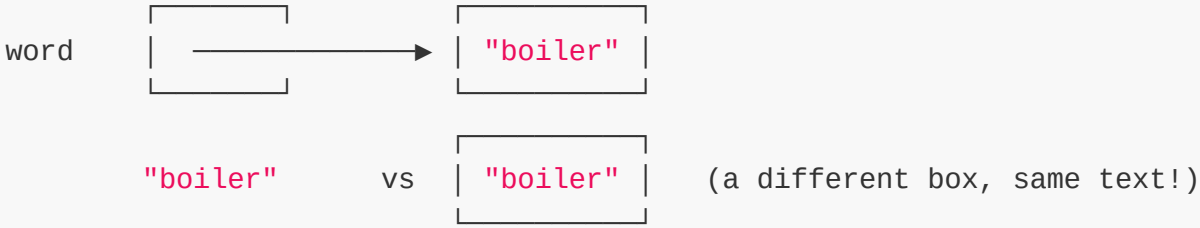
    if (word == "boiler")        // looks right, but is wrong
        IO.println("You got it!");
    if (word.equals("boiler"))   // is right
        IO.println("You got it!");
}
```

Why didn't this work?

We need to refine our abstraction.

A box containing a number is fine for "lower-case" number types, but "capital letter" types are different.

A String variable's box doesn't hold the text. Instead, it holds the **memory address** of the text.



`==` compares the **addresses**.

`.equals` compares the **text**.

Rule: Compare whole numbers with `==`; Strings with `.equals`.

Curious why `==` sometimes seems to work on Strings? There's a slide for after class at the very end.

Equality for Doubles

Part 3: the truth about decimals

Don't click run just yet on your laptop. What does this print?

```
void main() {
    IO.println(0.1 + 0.2);
}
```

▶ Expand

As we said before, this is not a bug, but a **truth about the machine**.

- a `double` stores numbers in binary
- 0.1 in binary is sort of like 1/3 in decimal ~ `0.33333...` which goes on forever
- It has to get cut off somewhere, and the leftover crumbs are what you're seeing.

The rule that follows: never use `==` on doubles.

```
if (0.1 + 0.2 == 0.3)     // false! never do this
```

Better approach: two `double`s are equal if they are really, really close together.

- For most purposes, like money, within `.0001` is a good threshold
- The closeness of two numbers `a` and `b` is the **absolute value** of their difference
- Use `Math.abs(a - b) < 0.0001`

Demo in class:

```
void main() {
    double sum = 0.1 + 0.2;
    IO.println(sum == 0.3);                    // false!
    IO.println(Math.abs(sum - 0.3) < 0.0001);  // true -- close enough
}
```

Casting

Question

We talked about casting already some yesterday.

What does this code (inside `void main()`) print?

```
int a = 7;
int b = 2;
IO.println((double) (a / b));
```

- 3.5
- 3.0
- 3
- 4
- It doesn't compile

Casting

Part 4: Forcing types: the cast

Java converts from `int → double` automatically (nothing is lost). The other direction loses the decimals, so Java makes **you** say it:

```
double price = 8.99;
int dollars = price;          // ERROR: possible lossy conversion
int dollars = (int) price;    // 8 (we said it's OK)
```

We can think of a cast as a promise to Java: "Trust me, I know what I'm doing here."

`(int)` **truncates** — chops the decimals, no rounding:

```
(int) 3.99    // 3
(int) -3.99   // -3
```

Want real rounding? `Math.round(3.99)` gives us `4`.

Cast Operator Properties

- Put type name in parentheses before expression
- The cast binds to the smallest expression possible, so we may need more parentheses:

```
void main() {
    double y = 14.6487;
    double x;
    x = (int) y / 3.0;      // compiles
    x = (int) (y / 3.0);    // compiles, but is it the same?
}
```

Fixing integer division

Our workaround used a `.0` literal in the source code: `35/8.0` is floating point division.

With two `int` *variables* there is nothing to hang a `.0` on; cast one of them:

```
void main() {
    int part = 3;
    int total = 8;
    IO.println(part / total);            // 0      (int / int)
    IO.println((double) part / total);   // 0.375  (double / int)
}
```

Note where the cast binds: it grabs just `part`, the smallest expression, which is exactly what we want here.

Pause and predict (poll)

Q. What does this print? `int a = 7; int b = 2; IO.println((double) (a / b));`

- **A.** 3.5
- **B.** 3.0
- **C.** 3
- **D.** 4
- **E.** It doesn't compile

Reference: The Numeric Family

Reference: the numeric family

Reference slide. Look up as needed; `int` and `double` remain the defaults.

Type	Bits	Range
`byte`	8	-128 to 127
`short`	16	-32,768 to 32,767
`int`	32	about ±2.1 billion
`long`	64	about ±9.2 quintillion (18 digits)
`float`	32	~7 significant digits
`double`	64	~15-16 significant digits

The exact boundaries are built in: `Integer.MAX_VALUE`, `Integer.MIN_VALUE`, `Double.MAX_VALUE` , etc. Every numeric type has them.

Big and small literals can use scientific notation: `3E8`, `6.02e23` (these are doubles).

That makes 8 primitives in all: these six, plus `char` (up next) and `boolean` .

The char type

A `char` is a number in disguise

A **`char`** is a single character in **single** quotes:

```
char first = 'B';
char digit = '7';
char space = ' ';
```

Strings are made of characters, but characters are different than Strings

- `'B'` is not the same as `"B"`
- `'7'` is not the same as `"7"` (or `7`)

The secret: Java stores every character as a **code number**.

char	`'0'`	`'A'`	`'B'`	`'a'`	`'b'`
code	48	65	66	97	98

So `'A'` *is* 65, as far as Java is concerned.

The set of possible `char` values is from 0 to 65535.

Buggy Program

Watch this fail

A natural way to get the letter *after* `'A'` is to add 1 to it:

```
void main() {
    char letter = 'A';
    IO.println("The next letter is: " + (letter + 1));
}
```

It compiles and runs. Here's the output:

▶ Expand

We wanted **B,** but we got **66** instead. Let's find out why.

char Operations

Doing math with characters

Because a `char` is a number, you can compute with it:

```
'A' + 1      // 66   (an int)
'z' - 'a'    // 25   (the distance between them)
```

Adding produces an **`int`**, not a `char`. To get a character back, **cast** it:

```
(char) ('A' + 1)    // 'B'
```

Why do we need to cast?

- Character values only go from 0 to 65535
- But we can add a much bigger integer like 99,999
- This won't fit back in a `char`

Characters also compare — and because a `char` is a whole number (a *primitive)*, `==` is safe here:

```
'a' < 'b'        // true   (97 < 98)
letter == 'A'    // true   (this is fine for char!)
```

Just like numbers, a `char` is treated as (upcast to) `String` for printing and concatenation

```
IO.println('A')   // prints A
"Hello" + '!'     // -> "Hello!"
```

Pause and predict (poll)

Q. What is `'f' - 'a'`?

- **A.** 4
- **B.** 5
- **C.** 6
- **D.** `'e'`
- **E.** You can't subtract characters

Poll Reveal: 'f' - 'a'

Question

What is `'f' - 'a'`?

- 4
- 5
- 6
- 'e'
- You can't subtract characters

Fixing the Program

Fix this Program:

The program we looked at printed **66** because `letter + 1` is arithmetic on code numbers, and arithmetic on a `char` gives us an **`int`**.

The bug in the program was that we need to tell Java we want a character back.

```
(char) (letter + 1)
```

Let's fix the program together:

```
void main() {
    char letter = 'A';
    IO.println("The next letter is: " + (letter + 1));
}
```

What happens if we don't put parentheses around `letter + 1`?

What if we say `((char) letter + 1)` instead?

Pause and predict (poll)

Q. What does `IO.println("Next: " + ((char) letter + 1));` print when `letter = 'A'`?

- **A.** Next: B
- **B.** Next: 'B'
- **C.** Next: 66
- **D.** Next: A1
- **E.** Doesn't compile

Poll Reveal: The Cast That Binds

Question

What does `IO.println("Next: " + ((char) letter + 1));` print when `letter = 'A'`?

- Next: B
- Next: 'B'
- Next: 66
- Next: A1
- Doesn't compile

Literals and Escape Sequences

`char` Literals

Remember a character literal is surrounded by single quotes, for example:

- `'A', 'a', 'x', '0', '3'`
- `'!', ',', '"', '&'`

Characters you can't type directly: escape sequences

Some characters need a backslash:

You write:	You get:
`\n`	newline
`\t`	tab
`\"`	a double quote (inside a `String`)
`\'`	a single quote (inside a `char`)
`\\`	a backslash
`\uXXXX`	any Unicode character by code

Example:

```
void main() {
    IO.println("Item:\tBURGER\nQty:\t2");
}
```

Character methods

Working with chars

Usually we don't need to think about code numbers.

Java's `Character` toolbox answers the common questions and returns a **`boolean`**

```
Character.isDigit('5')       // true
Character.isLetter('5')      // false
Character.toUpperCase('j')   // 'J'   (returns a char)
```

`Character` is a wrapper class, and one of its purposes is to contain useful methods.

- Similar to `Integer.parseInt("4")` for `int`
- Java has a full wrapper family: `Byte, Short, Integer, Long, Float, Double, Character, Boolean`.

Reference: Character

Reference: the `Character` toolbox

Reference slide You don't need to memorize these

Method	Returns	Example
`Character.isDigit(c)`	`boolean`	`'7'` → `true`
`Character.isLetter(c)`	`boolean`	`'A'` → `true`
`Character.isLetterOrDigit(c)`	`boolean`	`'!'` → `false`
`Character.isWhitespace(c)`	`boolean`	`' '` → `true`
`Character.isUpperCase(c)`	`boolean`	`'A'` → `true`
`Character.isLowerCase(c)`	`boolean`	`'a'` → `true`
`Character.toUpperCase(c)`	`char`	`'a'` → `'A'`
`Character.toLowerCase(c)`	`char`	`'A'` → `'a'`

Strings are characters

From characters to text: `String`

A **`String` is a sequence of `char`s**, numbered from **0**:

```
index:   0   1   2   3   4   5
         B   U   R   G   E   R
```

```
void main() {
    String s = "BURGER";
    s.length();    // 6     how many characters
    IO.println(s.charAt(0));   // 'B'   the char at a position
    IO.println(s.charAt(2));   // 'R'

    // IO.println(s.charAt(-1)); NO -- out of bounds
}
```

Valid positions are **0 to length − 1**.

`s.charAt(6)` **crashes** because there is no position 6:

```
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Index 6 out of b
```

A **run-time** error is a lot of text, but it is readable:

- It tells you which index and what length.

String operations

Looking inside a `String`

Here are several basic String operations we'll use again and again:

```
void main() {
    String s = "BURGER";
    IO.println(s.substring(1, 4));   // "URG"   index 1 up to — but NOT including — 4
    IO.println(s.substring(3));      // "GER"   from index 3 to the end
    IO.println(s.indexOf('R'));      // 2       position of the FIRST match
    IO.println(s.indexOf('Z'));      // -1      not found
    IO.println(s.contains("URG"));   // true
}
```

`substring(begin, end)` includes `begin`, excludes `end`

- the same "up to but not including" rule as `charAt`'s last index.

To compare text, use **`.equals`**, never `==`:

```
s.equals("BURGER")              // true
s.equalsIgnoreCase("burger")    // true
```

Reference: String

Reference: common `String` methods

Reference slide: look up as needed

All of these methods must be called following a string (variable) and a `.`

Example: `s.charAt(1)`

Method	Returns	Example on `"BURGER"`
`length()`	`int`	`6`
`charAt(i)`	`char`	`charAt(0)` → `'B'`
`substring(b, e)`	`String`	`substring(1,4)` → `"URG"`
`substring(b)`	`String`	`substring(3)` → `"GER"`
`indexOf(x)`	`int`	`indexOf('R')` → 2; missing → `-1`
`contains(x)`	`boolean`	`contains("URG")` → `true`
`startsWith(x)` / `endsWith(x)`	`boolean`	`startsWith("BUR")` → `true`
`toUpperCase()` / `toLowerCase()`	`String`	`toLowerCase()` → `"burger"`
`replace(a, b)`	`String`	`replace('R','X')` → `"BUXGEX"`
`trim()` / `strip()`	`String`	removes surrounding spaces
`equals(x)` / `equalsIgnoreCase(x)`	`boolean`	text comparison (not `==`)
`isEmpty()`	`boolean`	`"".isEmpty()` → `true`

Immutability

Strings don't change: immutability

A `String`'s characters are fixed the moment it's created.

In technical terms we say: Strings are **immutable**.

- There's no way to edit a character or extend a String in place.
- Every String operation hands you back a **brand-new** `String` and leaves the original alone:

```
String name = "ada";
name.toUpperCase();        // returns a new "ADA", then discards it
IO.println(name);         // still prints: ada
```



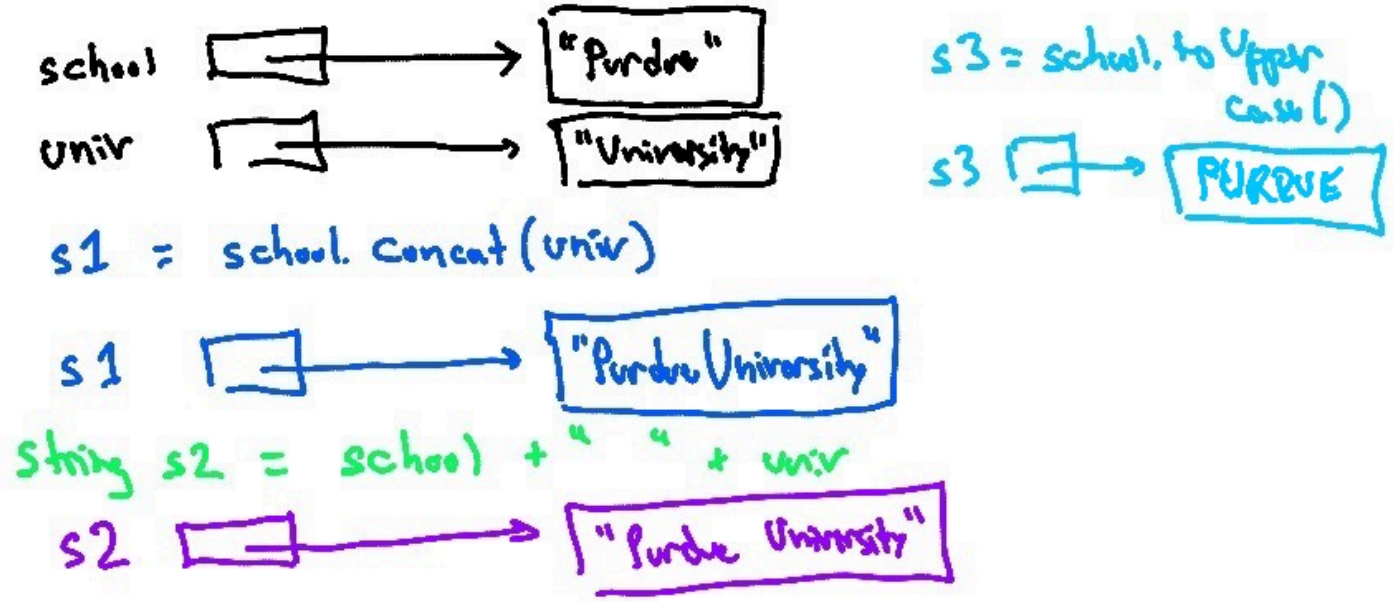
Immutability is a special property specific to Strings. When we meet other reference types and objects later, these will typically **not** be immutable.

Here's a larger example:

```
void main() {
    String school = "Purdue";
    String univ = "University";

    String s1 = school.concat(univ);   // "PurdueUniversity"
    String s2 = school + " " + univ;   // "Purdue University"
    String s3 = school.toUpperCase();  // "PURDUE"
}
```

Before we add `println` statements above, let's trace out what this does in detail:


school
"Purdue"
univ
"University"
s3 = school.toUpperCase()
s3
PURDUE
s1 = school.concat(univ)
s1
"Purdue University"
String s2 = school + " " + univ
s2
"Purdue University"

▶ Expand

Reassignment vs Mutation

Reassignment vs. changing in place

If you can't edit a String's letters, how do you "update" one?

You must **reassign** the variable using `=`. Remember: `=`-is-an-action!

```
void main() {
    String name = "purdue";
    name = name.toUpperCase();   // compute "PURDUE", then store it back in name
    IO.println(name);            // PURDUE
}
```

Nothing edited the original `"purdue"`. Java made a **new** String `"PURDUE"` and pointed `name` at it.

For reference types, such as String, there are two different meanings of change that we need to keep separate:

- **Changing in place:** editing the existing letters. *This is not possible for Strings.*
- **Reassignment:** compute a new String and store it back to the same variable name.
    - This is what we actually do.
    - The variable is a box containing a *different* address.

This distinction does not exist for primitive types (`int`, `double`, `char`, `boolean` etc) because the number is stored directly at the place in memory where a variable name refers.

There is no indirection for primitives.

Pause and predict (poll)

Q. What will this print?

```
String greeting = "hello";
greeting.replace("l", "L");
IO.println(greeting);
```

- **A.** nothing (blank)
- **B.** hello
- **C.** heLLo
- **D.** heLlo
- **E.** something else

Review: Strings

Question

What will the following code print out?

```
void main() {
    String greeting = "hello";
    greeting.replace("l", "L");
    System.out.println(greeting);
}
```

- nothing (blank)
- "hello"
- "heLLo"
- "heLlo"
- something else

Advanced: When == Seems to Work

Advanced: when == seems to work

Advanced material: Sometimes using `==` appears to work:

```
void main() {
    String a = "boiler";
    if (a == "boiler")                // true here (!)
        IO.println("== matched... this time");

    String b = IO.readln("Password? ");
    if (b == "boiler")                // never true for input
        IO.println("you will never see this");
    if (b.equals("boiler"))
        IO.println(".equals still works");
}
```

Why is that?

- The two different Strings must have had the same address.
- How? This is a compiler optimization by Java that only applies to String literals in your source code.